

Servicio Nacional de Aprendizaje
Tecnología en Desarrollo de Software

Manual de Estándares de Codificación del Proyecto

ejemplo01

Proyecto	ejemplo01 — Sistema de Gestión Empresarial
Versión	2.0
Fecha	03/08/2026
Autor	Jackson Darley Montoya Mercado
Ficha SENA	3118526
Tecnologías	Angular 22 / TypeScript / Angular Material / Tailwind CSS / RxJS

Control de versiones

Versión	Fecha	Autor	Descripción
1.0	03/08/2026	Jackson Darley Montoya Mercado	Versión inicial del manual de estándares.
2.0	03/08/2026	Jackson Darley Montoya Mercado	Reescritura completa basada en auditoría del código real.

Tabla de contenido

1. Introducción
2. Objetivos
3. Alcance
4. Arquitectura del proyecto
5. Organización del proyecto
6. Convenciones de nomenclatura
7. Estándares para componentes
8. Estándares para servicios
9. Estándares para modelos
10. Estándares para guards
11. Routing
12. Formularios
13. Buenas prácticas TypeScript
14. Buenas prácticas Angular
15. Buenas prácticas RxJS
16. Angular Material
17. Tailwind CSS
18. Calidad del código
19. Clean Code
20. SOLID
21. DRY
22. KISS
23. Seguridad
24. Rendimiento
25. Accesibilidad
26. Git
27. Convenciones de commits
28. Estrategia de ramas
29. Pull Requests
30. Revisión de código
31. Checklist antes de integrar cambios
32. Glosario
33. Referencias

34. Conclusiones

35. Recomendaciones futuras

1. Introducción

El presente documento establece los estándares de codificación oficiales del proyecto **ejemplo01**, un sistema de gestión empresarial desarrollado con **Angular 22** sobre **TypeScript**. El manual ha sido elaborado a partir de una **auditoría completa del código fuente real**, de modo que cada norma documentada refleja las convenciones, patrones y decisiones técnicas que el proyecto ya utiliza o que debería adoptar para su evolución.

Este documento es de aplicación obligatoria para todo desarrollador que realice cambios en el repositorio y sirve como guía oficial de desarrollo, revisión de código e incorporación de nuevos miembros al equipo.

2. Objetivos

- Garantizar la uniformidad del código fuente en todo el repositorio.
- Reducir la curva de aprendizaje de nuevos desarrolladores.
- Facilitar las revisiones de código mediante criterios objetivos.
- Documentar las decisiones arquitectónicas actuales del proyecto.
- Establecer un marco de calidad basado en principios de ingeniería de software.
- Servir como referencia oficial para la evolución futura del sistema.

3. Alcance

El manual aplica a la totalidad del código fuente del proyecto:

- Componentes (src/app/components/)
- Servicios (src/app/services/)
- Modelos (src/app/models/)
- Guardas (src/app/guards/)
- Rutas (src/app/app.routes.ts)
- Configuración de la aplicación (src/app/app.config.ts)
- Punto de entrada (src/main.ts)
- Estilos globales (src/styles.css)
- Configuración del proyecto (angular.json, tsconfig*.json, .prettierrc, .editorconfig, .postcssrc.json)

Nota: El proyecto es una aplicación 100 % frontend con datos simulados (mock). No utiliza backend, base de datos ni autenticación real. Cualquier referencia a estas tecnologías debe tratarse como recomendación futura, nunca como práctica existente.

4. Arquitectura del proyecto

El proyecto utiliza la **arquitectura standalone** de Angular (sin NgModules), con bootstrap directo de la aplicación y carga perezosa (lazy loading) en todas las rutas.

4.1 Flujo de autenticación

Flujo: LoginComponent -> AuthService -> localStorage -> Router

1. El usuario ingresa credenciales y presiona submit
2. AuthService.login() valida contra credenciales demo (admin / admin123)
3. Si son correctas: guarda auth_user + auth_token en localStorage
4. Navega a /dashboard
5. AuthGuard verifica isAuthenticated() antes de permitir rutas protegidas

4.2 Decisiones de diseño

Decisión	Justificación
Componentes standalone	Elimina NgModules innecesarios; simplifica imports y el árbol de dependencias.
Lazy loading con loadComponent	Reduce el bundle inicial; solo se carga el código de la ruta visitada.
Signals para estado reactivo	API moderna de Angular; evita la complejidad de Zone.js para estado local.
Inyección con inject()	Patrón funcional recomendado por Angular para componentes y guardas.
Servicios mock con Observable.of()	Simula latencia de red y mantiene la interfaz lista para un futuro HttpClient.
Barriles (barrel files)	Centralizan exportaciones y simplifican rutas de importación.

5. Organización del proyecto

```

ejemplo01/
|-- angular.json
|-- package.json
|-- tsconfig.json / tsconfig.app.json / tsconfig.spec.json
|-- .prettierrc / .editorconfig / .postcssrc.json
|-- docs/
|   |-- Estandares_Codificacion.pdf
|   |-- Manual_Estandares_Codificacion.md
|-- public/
|   |-- favicon.ico
|-- src/
|   |-- index.html
|   |-- main.ts
|   |-- styles.css
|   |-- app/
|       |-- app.ts
|       |-- app.config.ts
|       |-- app.routes.ts
|       |-- components/
|           |-- login/
|           |-- layout/
|           |-- dashboard/
|           |-- usuarios/
|           |-- productos/
|           |-- clientes/
|           |-- facturas/
|       |-- services/
|       |-- models/
|       |-- guards/

```

5.1 Responsabilidades por carpeta

Carpeta	Responsabilidad
src/app/components/	Componentes de presentación. Un componente por funcionalidad de negocio.
src/app/services/	Lógica de acceso a datos y reglas de negocio. Actualmente simula una API REST.
src/app/models/	Interfaces TypeScript que modelan las entidades del dominio del sistema.
src/app/guards/	Funciones de protección de rutas (autenticación).
docs/	Documentación oficial del proyecto.
public/	Archivos estáticos servidos en la raíz (favicon.ico).

Advertencia: Existe una carpeta ejemplo01/ duplicada en la raíz del repositorio que no está rastreada por Git (git ls-files ejemplo01 no devuelve resultados). Debe eliminarse o ignorarse explícitamente.

6. Convenciones de nomenclatura

Las siguientes convenciones han sido verificadas contra el código real del proyecto.

Elemento	Convención	Ejemplo real
Interfaces y tipos	PascalCase	Usuario, AuthResponse, EstadoFactura
Componentes	PascalCase + Component	LoginComponent, UsuariosComponent
Servicios	PascalCase + Service	AuthService, FacturasService
Guardas	camelCase + Guard	authGuard
Métodos	camelCase	obtenerTodos(), toggleSidebar()
Propiedades privadas	private readonly + camelCase	private readonly mockUsuarios
Señales	camelCase sin prefijo \$	usuarioActual, sidebarCollapsed, cargando
Variables inyectadas	camelCase + Service	private usuariosService = inject(UsuariosService)
Archivos / carpetas	kebab-case	usuario.model.ts, login.component.ts

Ejemplo correcto

```
export interface Usuario {  
  id: number;  
  nombre: string;  
}  
  
private usuariosService = inject(UsuariosService);  
usuariosFiltrados = signal<Usuario[]>([]);
```

Ejemplo incorrecto

```
export interface usuario {  
  ID: number;  
  nombre_completo: string;  
}  
  
private _service = inject(UsuariosService);  
$usuariosFiltrados = signal<Usuario[]>([]);
```

7. Estándares para componentes

7.1 Arquitectura standalone

```
@Component({
  selector: 'app-usuarios',
  standalone: true,
  imports: [CommonModule, FormsModule],
  templateUrl: './usuarios.component.html',
  styleUrls: ['./usuarios.component.css']
})
```

7.2 Tamaño máximo

Si un componente supera las **200 líneas**, debe separarse en archivos: .ts (lógica), .html (template) y .css (estilos).

Componente	Líneas	Acción recomendada
DashboardComponent	464	Extraer template y estilos
LayoutComponent	398	Extraer template y estilos
UsuariosComponent	384	Extraer estilos
LoginComponent	354	Extraer estilos
ProductosComponent	237	Extraer estilos

7.3 Control de flujo nativo

```
@for (usuario of usuariosFiltrados(); track usuario.id) {
  <tr><td>{{ usuario.nombre }}</td></tr>
} @empty {
  <tr><td colspan="7" class="empty-state">No se encontraron usuarios</td></tr>
}
```

Regla: Prohibido *ngIf y *ngFor para código nuevo. Toda iteración @for debe usar track con ID único.

7.4 Patrón de listado

```
export class UsuariosComponent implements OnInit {
  private usuariosService = inject(UsuariosService);
  usuarios: Usuario[] = [];
  usuariosFiltrados = signal<Usuario[]>([]);
  cargando = signal(true);

  ngOnInit(): void { this.cargarUsuarios(); }

  private cargarUsuarios(): void {
    this.cargando.set(true);
    this.usuariosService.obtenerTodos().subscribe({
      next: (data) => { this.usuarios = data; this.usuariosFiltrados.set(data); this.cargando.set(false); },
      error: () => this.cargando.set(false)
    });
  }
}
```

8. Estándares para servicios

8.1 Definición y barril

```
@Injectable({ providedIn: 'root' })
export class UsuariosService { ... }

// services/index.ts
export { UsuariosService } from './usuarios.service';
```

8.2 Interfaz CRUD uniforme

Los 4 servicios de negocio (UsuariosService, ProductosService, ClientesService, FacturasService) implementan la misma firma CRUD:

```
obtenerTodos(): Observable<Modelo[]>;
obtenerPorId(id: number): Observable<Modelo | undefined>;
crear(entity: Omit<Modelo, 'id'>): Observable<Modelo>;
actualizar(entity: Modelo): Observable<Modelo>;
eliminar(id: number): Observable<boolean>;
```

8.3 Simulación de latencia

Operación	Retardo	Método
Lista completa	500 ms	obtenerTodos()
Búsqueda por ID	300 ms	obtenerPorId(id)
Autenticación	800 ms	login(credenciales)
Operaciones CRUD	500 ms	crear, actualizar
Eliminación	300 ms	eliminar(id)

8.4 Tipado estricto

```
// Debe corregirse en FacturasService:
actualizarEstado(id: number, estado: string) // Incorrecto
actualizarEstado(id: number, estado: EstadoFactura) // Correcto
```

9. Estándares para modelos

9.1 Documentación @property

```
/**
 * @property id - Identificador numérico único del usuario
 * @property nombre - Nombre completo del usuario
 */
export interface Usuario {
  id: number;
  nombre: string;
  email: string;
}
```

9.2 Tipos unión para estados

```
export type EstadoFactura = 'PENDIENTE' | 'PAGADA' | 'ANULADA' | 'VENCIDA';
```

9.3 Composición sobre herencia

```
export interface Factura {
  cliente: Cliente;
  detalles: DetalleFactura[];
  estado: EstadoFactura;
}
```

9.4 Barriles de importación

```
// models/index.ts
export { type Usuario, type Credenciales, type AuthResponse } from './usuario.model';
export { type Producto } from './producto.model';

// Uso correcto en componentes
import { Usuario, Credenciales, AuthResponse } from '../models';
```

10. Estándares para guards

Las guardas deben implementarse como **funciones** (CanActivateFn), no como clases.

```
export const authGuard: CanActivateFn = (): boolean | UrlTree => {  
  const authService = inject(AuthService);  
  const router = inject(Router);  
  
  if (authService.isAuthenticated()) {  
    return true; // Permite el acceso  
  }  
  return router.parseUrl('/login'); // Redirige al login  
};
```

Regla: Reglas: usar inject() dentro de la función, retornar true para permitir, retornar UrlTree con router.parseUrl() para redirigir, y verificar isAuthenticated() del servicio.

11. Routing

11.1 Carga perezosa obligatoria

```
{
  path: 'dashboard',
  loadComponent: () =>
    import('./components/dashboard/dashboard.component').then(m => m.DashboardComponent),
  title: 'Dashboard - Sistema Empresarial'
}
```

11.2 Títulos de página obligatorios

Ruta	Título
/login	Inicio de Sesión - Sistema Empresarial
/dashboard	Dashboard - Sistema Empresarial
/usuarios	Usuarios - Sistema Empresarial
/productos	Productos - Sistema Empresarial
/clientes	Clientes - Sistema Empresarial
/facturas	Facturas - Sistema Empresarial
/**	redirectTo: '/dashboard'

11.3 Protección de rutas

```
{
  path: '',
  canActivate: [authGuard],
  loadComponent: () =>
    import('./components/layout/layout.component').then(m => m.LayoutComponent),
  children: [ /* rutas hijas del dashboard */ ]
}
```

12. Formularios

El proyecto utiliza **formularios template-driven** con FormsModule y ngModel.

```
<form (ngSubmit)="onSubmit()" class="login-form">
  <input
    id="usuario"
    type="text"
    [(ngModel)]="credenciales.usuario"
    name="usuario"
    required
    autocomplete="username"
  />
</form>
```

Regla: Reglas: todo control con ngModel debe definir name; los formularios de login deben incluir autocomplete; validar en onSubmit() y mostrar errorMessage.

Recomendación: No se usa ReactiveFormsModule en el código actual. Los formularios reactivos (FormGroup, FormControl) se consideran una recomendación futura para formularios complejos.

13. Buenas prácticas TypeScript

13.1 Opciones estrictas (no desactivar)

Opción	Valor
noImplicitOverride	true
noPropertyAccessFromIndexSignature	true
noImplicitReturns	true
noFallthroughCasesInSwitch	true
isolatedModules	true
experimentalDecorators	true
target	ES2022

13.2 Tipado explícito y readonly

```
// Correcto
private readonly mockUsuarios: Usuario[] = [/* ... */];
credenciales: Credenciales = { usuario: '', contraseña: '' };

// Incorrecto
private mockUsuarios = [];
credenciales = {};
```

13.3 Omit en creación

```
crear(cliente: Omit<Cliente, 'id'>): Observable<Cliente> // Correcto
crear(cliente: Cliente): Observable<Cliente>             // Incorrecto
```

14. Buenas prácticas Angular

14.1 Inyección con inject()

```
// Correcto
private facturasService = inject(FacturasService);
private router = inject(Router);

// Incorrecto
constructor(private facturasService: FacturasService) {}
```

14.2 Señales (Signals)

```
private usuarioActual = signal<Usuario | null>(null);
readonly isAuthenticated = computed(() => this.usuarioActual() !== null);
sidebarCollapsed = signal(false);
```

14.3 Exponer Observables desde servicios

```
obtenerTodos(): Observable<Usuario[]> {
  return of([...this.mockUsuarios]).pipe(delay(500));
}
```

15. Buenas prácticas RxJS

15.1 Suscripciones con {next, error}

```
this.authService.login(this.credenciales).subscribe({
  next: (respuesta) => {
    this.cargando = false;
    if (respuesta.success) { this.router.navigate(['/dashboard']); }
  },
  error: () => {
    this.cargando = false;
    this.errorMensaje = 'Error de conexión. Intente nuevamente.';
  }
});
```

15.2 Manejo de errores obligatorio

Corrección requerida: El componente LoginComponent es el único que cumple la regla actualmente; los listados usan `error: () => this.cargando.set(false)` sin feedback. Toda suscripción DEBE manejar error con feedback al usuario o al menos log.

Nota: Los observables actuales provienen de `of()` y no producen errores en la práctica. Sin embargo, el handler error es obligatorio para preparar la migración a `HttpClient`.

Recomendación futura: Usar `takeUntilDestroyed()` cuando se introduzcan streams infinitos o suscripciones persistentes.

16. Angular Material

Estado actual: @angular/material y @angular/cdk están instalados y styles.css contiene reglas de personalización (.mat-toolbar, .mat-mdc-card, .mat-mdc-header-row, .mat-mdc-form-field). Sin embargo, **ningún template importa o utiliza componentes de Angular Material.**

Regla aplicable

No es obligatorio usar Angular Material. Los componentes de UI del proyecto son HTML/CSS nativos personalizados. Las reglas CSS existentes en styles.css se conservan para preparar su adopción.

Recomendación futura: Si se adoptan componentes Material, deberá definirse un tema oficial (indigo #1a237e, teal #00897b, rojo #d32f2f) y documentar su uso en una nueva sección del manual.

17. Tailwind CSS

Estado actual: Tailwind 4 está configurado (@import "tailwindcss" en styles.css, plugin en .postcssrc.json). Los templates **no utilizan clases utilitarias**; todo el estilo se basa en CSS scoped personalizado por componente.

Regla aplicable

No introducir clases utilitarias de Tailwind en templates sin aprobación de revisión. Mantener la consistencia con el sistema de estilos actual: CSS scoped en el componente o estilos globales en styles.css.

Recomendación futura: Adoptar gradualmente Tailwind para utilidades puntuales (layout, espaciado, responsive) pero definiendo primero una guía de tokens de diseño.

18. Calidad del código

18.1 Formato

Regla	Configuración
Indentación	2 espacios
Comillas	Simples
Ancho de línea	100 columnas
Salto de línea final	Obligatorio
Espacios al final de línea	Prohibidos

Nota: Fuentes de verdad: .prettierrc y .editorconfig.

18.2 Documentación interna (banner)

```
/**
 * =====
 * SERVICIO: ProductosService
 * =====
 * Propósito: Servicio que simula (Mock) las operaciones CRUD
 * para la gestión del inventario de productos.
 * =====
 */
```

18.3 Idioma

Todo el código (identificadores, comentarios, mensajes UI) se escribe en **español**.

```
// Correcto
cargarUsuarios(), errorMessage, 'Por favor ingrese usuario y contraseña'

// Incorrecto
loadUsers(), errorMessage, 'Please enter your credentials'
```

19. Clean Code

- **Nombres descriptivos:** los métodos dicen qué hacen (obtenerTodos(), toggleSidebar(), cerrarSesion()).
- **Funciones pequeñas:** cada método tiene una única responsabilidad (cargarDatos(), actualizarTarjetas(), buscar()).
- **Sin magia:** los datos mock documentan su propósito con comentarios.
- **Legibilidad:** los templates usan clases con nombres semánticos (login-card, stat-card, table-container).
- **Manejo defensivo:** try/catch al parsear localStorage (auth.service.ts).

Ejemplo correcto

```
private cargarSesion(): void {
  if (isPlatformBrowser(this.platformId)) {
    const sesionGuardada = localStorage.getItem('auth_user');
    if (sesionGuardada) {
      try {
        this.usuarioActual.set(JSON.parse(sesionGuardada));
      } catch {
        localStorage.removeItem('auth_user');
      }
    }
  }
}
```

20. SOLID

Principio	Aplicación en el proyecto
S - Responsabilidad única	Cada servicio gestiona una sola entidad (UsuariosService → usuarios).
O - Abierto/Cerrado	Los modelos extienden comportamiento mediante composición, no modificación.
L - Sustitución	Las interfaces tipan contratos; los servicios devuelven exactamente el modelo.
I - Segregación	Los componentes consumen solo los métodos que necesitan.
D - Inversión	Los componentes dependen de servicios, no de implementaciones concretas.

Recomendación: Para reforzar D, considerar definir una interfaz de contrato por servicio (ej. CrudService) que abstraiga las operaciones CRUD y permita reemplazar el mock por HttpClient sin tocar componentes.

21. DRY (Don't Repeat Yourself)

Hallazgo real: existe duplicidad verificada en el proyecto:

- **Estilos copiados:** los componentes productos, clientes y facturas comparten bloques CSS idénticos (page, btn-primary, table, badge, spinner).
- **Patrón cargar:** se repite en los 4 listados con la misma estructura (cargando.set(true) → subscribe → set()).
- **Patrón buscar:** filtrado con toLowerCase().includes() repetido en usuarios, productos, clientes y facturas.
- **Botones y acciones:** btn-edit / btn-delete con alert()/confirm() idénticos.

Plan de corrección

- Centralizar las clases comunes (tablas, botones, badges, estados vacíos, loader) en styles.css.
- Evaluar la creación de una utilidad o función compartida para la carga y filtrado de listados.

22. KISS (Keep It Simple, Stupid)

- Los servicios exponen observables pequeños y predecibles.
- Las señales simplifican el flujo de datos sin librerías de estado externas.
- El flujo de autenticación es directo y comprensible.
- El proyecto evita complejidad innecesaria: sin NgRx, sin servicios HTTP, sin formularios reactivos hasta que se necesiten.

23. Seguridad

23.1 Protección de rutas

```
if (authService.isAuthenticated()) {  
  return true;  
}  
return router.parseUrl('/login');
```

23.2 Persistencia segura en SSR

```
// Correcto  
if (isPlatformBrowser(this.platformId)) {  
  localStorage.setItem('auth_user', JSON.stringify(this.mockUsuario));  
}  
  
// Incorrecto  
localStorage.setItem('auth_user', JSON.stringify(this.mockUsuario));
```

23.3 Credenciales de demostración

Advertencia: Las credenciales admin / admin123 están hardcodeadas en AuthService con fines educativos. No deben utilizarse en producción. Si el sistema se conecta a un backend real, estas credenciales deben eliminarse y sustituirse por autenticación segura (tokens, hashing, HTTPS).

24. Rendimiento

- **Presupuestos de bundle:** angular.json define budgets de producción de 500 kB (advertencia) / 1 MB (error) para el bundle inicial.
- **Carga perezosa:** todas las rutas cargan sus componentes bajo demanda, lo que reduce el bundle inicial.
- **Inmutabilidad de datos mock:** los arrays internos (mockProductos, mockClientes) se copian con spread [...] lo que evita efectos secundarios.
- **Pipes puros:** usar currency y date (pipes puros de Angular) ya es correcto.

Recomendación: Evitar llamadas redundantes: DashboardComponent llama a los 4 servicios en ngOnInit; considerar forkJoin cuando sea necesario.

25. Accesibilidad

Prácticas presentes en el proyecto:

- label con atributo for vinculado al id del input en LoginComponent.
- autocomplete="username" y autocomplete="current-password" en el formulario de login.
- title en botones de icono (title="Editar", title="Eliminar", title="Cerrar sesión").
- lang="es" en index.html.

Recomendaciones

- Añadir aria-label a los botones que solo contienen iconos.
- Asegurar contraste suficiente (verificar pares como #1a237e sobre blanco).
- Probar navegación con teclado en tablas y modales.

26. Git

```
# Ver estado del repositorio
git status

# Ver cambios realizados
git diff

# Añadir archivos al área de staging
git add <archivo>

# Crear un commit con mensaje descriptivo
git commit -m "fix: corregir validación de login"

# Subir cambios a la rama remota
git push origin main
```

Historial verificado del repositorio

Commit	Descripción
9ad3b15	Initial commit: Proyecto AG - Angular application
1de5d13	docs: agrega estandares de codificacion del proyecto
3b746c6	docs: agregar manual de estandares de codificacion en carpeta docs

27. Convenciones de commits

<tipo>: <descripción en presente, minúsculas>

Prefijo	Uso	Ejemplo
feat:	Nueva funcionalidad	feat: agregar módulo de facturación
fix:	Corrección de errores	fix: corregir validación de login
docs:	Cambios en documentación	docs: actualizar manual de estándares
refactor:	Mejora sin cambiar comportamiento	refactor: simplificar servicio de usuarios
style:	Formato sin afectar lógica	style: aplicar formato prettier
test:	Pruebas	test: agregar pruebas de auth
chore:	Mantenimiento	chore: actualizar dependencias

```
# Correcto
git commit -m "docs: agregar manual de estándares de codificación"

# Incorrecto
git commit -m "cambios"
```

28. Estrategia de ramas

main

```
| -- feature/nombre-funcionalidad  
|-- hotfix/nombre-correccion
```

Flujo recomendado:

1. git checkout -b feature/nombre-funcionalidad
2. Realizar commits pequeños y descriptivos
3. Abrir un Pull Request hacia main
4. Integrar tras la aprobación del revisor

29. Pull Requests

- **Descripción clara:** qué se cambia, por qué, y cómo se probó.
- **Commits atómicos:** un commit por cambio lógico.
- **Sin errores de build:** ejecutar ng build antes de abrir o fusionar.
- **Sin conflictos:** ejecutar git pull --rebase origin main antes de fusionar.

30. Revisión de código

- Cumple los estándares de nomenclatura (sección 6).
- Usa componentes standalone y lazy loading.
- Usa inject() y señales.
- Maneja errores en todas las suscripciones.
- No introduce duplicidad (DRY).
- Respeta los límites de tamaño de componente.
- Formato correcto con Prettier.
- Documentación con banner y @property.

31. Checklist antes de integrar cambios

- ✓ ng build compila sin errores.
- ✓ ng test pasa (cuando existan pruebas).
- ✓ Formato aplicado con Prettier.
- ✓ No hay clases utilitarias de Tailwind sin aprobación.
- ✓ No se desactivaron opciones estrictas del tsconfig.json.
- ✓ Toda suscripción tiene handler error.
- ✓ Los archivos nuevos incluyen banner de documentación.
- ✓ Se actualizó este manual si el estándar cambió.
- ✓ git status muestra solo los archivos intencionales.
- ✓ El commit usa el prefijo correcto.

Regla: Marcar todos los elementos antes de abrir un Pull Request.

32. Glosario

Término	Definición
Standalone	Arquitectura Angular sin NgModules que usa imports directamente en el componente.
Signal	Primitiva reactiva de Angular para estado (signal(), computed()).
Lazy loading	Carga diferida de código bajo demanda mediante loadComponent.
Barrel file	Archivo index.ts que re-exporta módulos y simplifica importaciones.
Mock	Datos simulados que reemplazan un servicio real de backend.
Guard	Función (CanActivateFn) que protege rutas en Angular Router.
CRUD	Create, Read, Update, Delete — operaciones básicas de persistencia.

33. Referencias

Tema	Referencia
Angular	https://angular.dev
Angular Signals	https://angular.dev/guide/signals
Angular Routing	https://angular.dev/guide/routing
Angular Control Flow	https://angular.dev/guide/templates/control-flow
Lazy Loading	https://angular.dev/guide/ngmodules/lazy-loading
Conventional Commits	https://www.conventionalcommits.org
Prettier	https://prettier.io
Vitest	https://vitest.dev
RxJS	https://rxjs.dev

34. Conclusiones

El proyecto ejemplo01 cuenta con una arquitectura Angular moderna y limpia: standalone, lazy loading, señales, inyección funcional y control de flujo nativo. La documentación interna del código es ejemplar, con banners descriptivos en todos los archivos. El proyecto mantiene coherencia entre los módulos de negocio (usuarios, productos, clientes, facturas), servicios mock y modelos tipados.

Este manual establece el marco de estándares que el proyecto ya practica en su mayoría y define las correcciones necesarias: tipar correctamente los parámetros de estado, manejar errores en todas las suscripciones, separar componentes grandes, centralizar estilos duplicados y eliminar la carpeta ejemplo01/no rastreada.

La aplicación de este manual convierte el repositorio en una base sólida para el crecimiento del sistema, ya sea incorporando un backend real, pruebas automatizadas o nuevos módulos de negocio.

35. Recomendaciones futuras

Las siguientes recomendaciones no describen el estado actual del proyecto y solo deberán incorporarse cuando se planifique su implementación:

Área	Recomendación
Backend	Conectar los servicios mock a una API REST con HttpClient e interceptores.
Entornos	Crear environments/environment.ts (dev) y environment.prod.ts.
Pruebas	Escribir pruebas unitarias con Vitest (ya configurado en tsconfig.spec.json).
Formularios reactivos	Migrar a ReactiveFormsModule cuando existan formularios complejos.
Manejo de errores global	Implementar un ErrorHandler global y mensajes de usuario estandarizados.
Componentes compartidos	Crear una carpeta shared/ para botones, tablas y badges reutilizables.
Limpieza	Eliminar la carpeta ejemplo01/ duplicada y verificar git status limpio.
Assets	Crear src/assets/ y añadir las imágenes referenciadas por Producto (laptop.png, etc.).
Angular Material	Adoptar componentes Material con un tema oficial cuando el sistema lo requiera.
Tailwind	Definir tokens de diseño y adoptar utilidades para layout y responsive.

Documento generado a partir de la auditoría completa del repositorio ejemplo01. Ficha SENA 3118526 — Autor: Jackson Darley Montoya Mercado